

# Pacman Search Assignment

Group: 24I-3166, 24I-3106, 24I-3015 · Section 5B

## 1. Algorithms Implemented

The table below summarises the frontier data structure and ordering rule used by each algorithm before the individual descriptions.

| Algorithm | Frontier Structure | Expansion Order             |
|-----------|--------------------|-----------------------------|
| DFS       | Stack (LIFO)       | Most recently added         |
| BFS       | Queue (FIFO)       | Earliest added              |
| UCS       | Priority queue     | Lowest $g(n)$               |
| GBFS      | Priority queue     | Lowest $h(n)$               |
| A*        | Priority queue     | Lowest $f(n) = g(n) + h(n)$ |

### 1.1 Depth-First Search (DFS)

DFS is implemented using a LIFO stack as the frontier, combined with an explored set. It expands the most recently generated node first, so it commits early to a single branch and only backtracks once that branch is exhausted or leads to a dead end. This means DFS returns the first goal it finds, which can be far longer than the shortest path.

### 1.2 Breadth-First Search (BFS)

BFS is implemented using a FIFO queue as the frontier, together with an explored set. It expands states in order of increasing depth, so the goal is reached via the fewest possible steps. The cost of this guarantee is memory, since BFS must hold an entire frontier layer at once.

### 1.3 Uniform-Cost Search (UCS)

UCS uses a priority queue ordered by accumulated path cost  $g(n)$ . A cheaper path to a state already on the frontier is pushed as a new entry, and the explored-set check discards the stale entry later. This guarantees that the first time a state is popped, it is reached via its cheapest known path.

### 1.4 Greedy Best-First Search (GBFS)

GBFS uses a priority queue ordered only by the heuristic value  $h(n)$ , ignoring  $g(n)$  entirely. It tends to move directly toward the goal and often expands fewer nodes, but has no way of recognising when it has committed to an expensive route.

### 1.5 A\* Search

A\* uses a priority queue ordered by  $f(n) = g(n) + h(n)$ . With an admissible and consistent heuristic, A\* is guaranteed to find the optimal solution while typically expanding fewer nodes than uninformed search.

### 1.6 Summary

| Algorithm | One-line Description                                      |
|-----------|-----------------------------------------------------------|
| DFS       | Explores as deep as possible before backtracking          |
| BFS       | Explores level by level, nearest states first             |
| UCS       | Explores lowest accumulated cost first                    |
| GBFS      | Explores state that looks closest to the goal             |
| A*        | Explores lowest cost-so-far plus estimated remaining cost |

## 2. State Space Representation

The table below summarises the state representation and heuristic used for each search problem.

| Problem                     | State Representation                                    | Heuristic Used                                        |
|-----------------------------|---------------------------------------------------------|-------------------------------------------------------|
| Position Search (Tasks 1-5) | Pacman (x, y) position                                  | Manhattan / Euclidean (GBFS, A*)                      |
| Corners Problem             | (position, visited_tuple)<br>visited_tuple = 4 booleans | Distance to nearest corner + MST of remaining corners |
| Food Search Problem         | (position, food_grid)                                   | Max maze distance to any remaining food dot           |

The corners state adds a 4-bit visited tuple, multiplying the state count by up to 16 relative to plain position search. The food search state grows exponentially with the number of remaining food dots, since any subset of dots is a distinct state.

### 3. Complexity Analysis

Here  $b$  is the branching factor,  $d$  is the depth of the shallowest goal,  $m$  is the maximum depth of the search tree, and  $C^*$  is the cost of the optimal solution.

| Algorithm | Complete? | Optimal?        | Time Complexity    | Space Complexity   |
|-----------|-----------|-----------------|--------------------|--------------------|
| DFS       | Yes       | No              | $O(b^m)$           | $O(b^m)$           |
| BFS       | Yes       | Yes (unit cost) | $O(b^d)$           | $O(b^d)$           |
| UCS       | Yes       | Yes             | $O(b^{(1+C^*/e)})$ | $O(b^{(1+C^*/e)})$ |
| GBFS      | Yes*      | No              | $O(b^m)$           | $O(b^m)$           |
| A*        | Yes       | Yes             | $O(b^d)$           | $O(b^d)$           |

\* GBFS is complete here only because an explored set prevents infinite loops; without one it can loop indefinitely on graphs with cycles.

### 3.1 Discussion

The uninformed algorithms (DFS, BFS, UCS) differ mainly in the order they expand the frontier, and this ordering directly determines their optimality guarantees. DFS trades optimality for low memory use, since it only needs to store one path at a time. BFS and UCS trade memory for optimality: both must keep an entire frontier layer or a full priority queue in memory, but in return they guarantee the shortest or cheapest solution respectively.

The informed algorithms (GBFS, A\*) both rely on a heuristic function to estimate the remaining distance to the goal, but they use that estimate differently. GBFS treats the heuristic as the only signal worth following, which can expand fewer nodes but sacrifices any guarantee of optimality. A\* combines the heuristic with the accumulated path cost, which preserves optimality provided the heuristic is admissible and consistent, at the cost of exploring a somewhat larger portion of the search space than GBFS alone.

In general, the tightness of the heuristic has a direct effect on A\*'s practical performance: a heuristic closer to the true remaining cost narrows the effective branching factor of the search, reducing the number of nodes expanded without affecting the correctness of the result.

## 4. Practical Complexity by Problem Type

The worst-case bounds above rarely occur in practice. The table below summarises the practical behaviour of each problem type given the branching factor bound of 4 for Pacman moves.

| Problem Type    | Effective Branching             | Dominant Cost Factor      |
|-----------------|---------------------------------|---------------------------|
| Position Search | Low, near-constant ( $\leq 4$ ) | Maze size, solution depth |

|                 |                            |                         |
|-----------------|----------------------------|-------------------------|
| Corners Problem | Moderate (state space x16) | Heuristic tightness     |
| Food Search     | High (exponential in dots) | Heuristic admissibility |

DFS's space advantage grows with maze size, since its memory use scales with path depth rather than the number of states at a given depth. BFS and UCS can see memory use grow substantially on mazes with many branch points, since every branch must be represented in the frontier before any of them is expanded further.

For GBFS and A\*, node expansion depends on how well the heuristic approximates the true remaining cost. A\* never sacrifices correctness for speed, whereas GBFS can be misled by a heuristic that underestimates the cost of a particular route, since it never revisits its assumptions about accumulated cost once committed to a direction.

## 5. Execution Trace Logging

Each algorithm writes a per-iteration execution trace to a CSV file. The columns recorded are shown below.

| Column                  | Description                              |
|-------------------------|------------------------------------------|
| iteration               | Sequential expansion number              |
| expanded_state          | State popped from the frontier           |
| parent                  | Predecessor state                        |
| action                  | Action taken to reach this state         |
| generated_successors    | Number of successors generated           |
| frontier_before / after | Frontier size before and after expansion |

|          |                                |
|----------|--------------------------------|
| explored | Size of the explored set       |
| g, h, f  | Cost values (where applicable) |

For the food search problem, states are logged as a position paired with the remaining food count rather than the full grid, and the explored set is logged as a count rather than a full listing, since serialising either in full would be far more expensive than the search itself.